

Text_Analysis_literature_review

November 28, 2025

```
[97]: import re
      from pathlib import Path
      from typing import List, Dict, Any
      import pandas as pd
      from tqdm import tqdm
      from pdfminer.high_level import extract_text
      from pdfminer.high_level import extract_text as pdfminer_extract_text
```

```
[98]: import logging
      logging.getLogger("pdfminer").setLevel(logging.ERROR)
```

1 Risk-of-Bias Tool Usage / Quality assessment in Consumer Systematic Literature Reviews

This notebook analyzes whether systematic reviews in the domain of **consumer* systematic literature review* food** make use of quality assessment or risk-of-bias, including tools such as:

- Cochrane RoB tool / Revised RoB tool (RoB2) / ROBINS-I / ROBINS-E
- Newcastle–Ottawa Scale (NOS)
- QUADAS-2
- Jadad Score - MIXED METHODS APPRAISAL TOOL (MMAT) - Quality assessment with diverse studies(QuADS) and Quality assessment in general

QUERY ON SCOPUS

TITLE-ABS-KEY (consumer* systematic literature review* food) - 933 articles found - 633 downloaded

```
[99]: # Database
      BASE_FOLDER = Path("/Users/martabonioli/Library/CloudStorage/Dropbox/
      ↪Smart_labels/Response_letter_tothe_editor/Marta/relevant papers/
      ↪scopus_consumer*systematicLitRev*food")
      OUTPUT_CSV = BASE_FOLDER /
      ↪"RoB_QualityAss_analysis_scopus_633_keywords_271125_5.csv"
```

1.0.1 Summary of what this script does

- Defines the dictionary of search terms

- Extracts text from each PDF until “References”
- Searches for each term of the dictionary in the relevant text (Only front-matter up to “Keywords” (read_pdf_until_keywords) for “Meta analysis” and “Trends in Food Science & Technology”)
- Saves results:

****_found**** columns show True or False if a match was found.

****_sentences**** columns store up to 5 example sentences containing the term.

- Exports the final table as a CSV file.

```
[100]: # TOOL DICTIONARY
# Each key represents a tool or term to be searched for in the PDFs.
# Each value is a list of possible textual variations that might appear in the
# papers.
tool_dict: Dict[str, List[str]] = {
    "Quality assessment": [
        "quality assessment", "quality assessments",
        "assessment of quality",
        "study quality",
        "quality of the study",
        "quality of the studies",
        "quality of literature", "quality of the literature",
        "quality of the paper", "quality of the papers",
        "quality of the article", "quality of the articles",
        "quality of included study", "quality of included studies",
        "discuss the quality", "quality of included studies",
        "quality criteria"
    ],
    "Impact Factor" : ["impact factor", "citescore", "cite score", "SNIP",
        "source normalized impact per paper", "scimago"],
    "PRISMA": [
        "prisma",
        "prisma flow diagram",
        "prisma 2020",
        "preferred reporting items for systematic reviews and meta-analyses",
        "prisma statement"
    ],
    "Peer review": [
        "peer review", "peer-review",
        "peer reviews", "peer-reviews",
        "peer reviewed", "peer-reviewed"
    ],
    "Risk of Bias": [
```

```

    "rob",
    "risk of bias",
    "risk of bias tool",
    "risk of bias tools",
    "risk of bias assessment",
    "bias assessment",
    "selection bias",
    "publication bias",
    "mmat",
    "mixed methods appraisal tool",
    "robins-i",
    "robins i",
    "robins-e",
    "robins e",
    "newcastle-ottawa",
    "newcastle ottawa",
    "nos",
    "nos scale",
    "quadas-2",
    "quadas 2",
    "jadad",
    "jadad score"
],

"Agreement": [
    "agreement on study", "agreement on studies",
    "agreement on the study", "agreement on the studies",
    "agreement on article", "agreement on articles",
    "agreement on the article", "agreement on the articles",
    "agreement on literature", "agreement on the literature", "agreement_
↪about the literature", "agreement regarding the literature"
    "agreement between reviewers", "agreement among reviewers",
    "agreement between authors", "agreement among authors",
    "reviewer agreement", "reviewers agreement",
    "authors agreement",
    "inter-reviewer agreement", "inter-reviewers agreement",
    "agree upon", "agreed upon",
    "consensus",
    "discussion between"
],

"Disagreement": [
    "disagreement on study", "disagreement on studies",
    "disagreement on the study", "disagreement on the studies",
    "disagreement on article", "disagreement on articles",
    "disagreement on the article", "disagreement on the articles",
    "disagreement on literature", "disagreement on the literature",_
↪"agreement about the literature", "agreement regarding the literature"

```

```

        "disagreement between reviewers", "disagreement among reviewers",
        "disagreement between authors", "disagreement among authors",
        "reviewer disagreement", "reviewers disagreement", "reviewer_
↪disagreements", "reviewers disagreements",
        "authors disagreement", "authors disagreements",
        "inter-reviewer disagreement", "inter-reviewers disagreement",
        "disagree upon", "disagreed upon",
        "discrepancies", "discrepancy"
    ],

    # English language
    "English": [
        "english",
        "en"
    ],
    # The following terms will be searched only in the front part
    # of the paper (title + abstract + keywords)
    "Meta-analysis": [
        "meta analysis", "meta-analysis", "metaanalysis",
        "meta analysi", "meta-analysi", "metaanalysi",
        "-metaanalysis-"
    ],
    "Trends in Food Science & Technology": [
        "trends in food science & technology",
        "trends in food science and technology"
    ]
}

```

[101]: # UTILITY FUNCTIONS

```

def normalize_spaces(text: str) -> str:
    """
    Normalize whitespace and hyphen characters so that pdfminer text
    extraction does not lose words connected by different Unicode dashes.
    """
    import re
    # unify whitespace
    text = re.sub(r"\s+", " ", text or "")
    # normalize dash characters
    text = (
        text.replace("\u00AD", "") # soft hyphen (remove)
        .replace("\u2010", "-") # hyphen
        .replace("\u2011", "-") # non-breaking hyphen
        .replace("\u2012", "-") # figure dash
        .replace("\u2013", "-") # en dash
        .replace("\u2014", "-") # em dash
    )

```

```

        .replace("\u2015", "-") # horizontal bar
    )
    return text.strip()

def read_pdf_text(path: Path) -> str:
    """
    Extract the full text from a PDF file using pdfminer,
    cut 'References' to exclude that section.
    """
    try:
        from pdfminer.high_level import extract_text
        # 1) testo grezzo dal PDF
        txt_raw = extract_text(str(path)) or ""
        # 2) taglia alla sezione 'References'
        txt_no_refs = cut_at_references(txt_raw)
        # 3) normalizza spazi e trattini
        return normalize_spaces(txt_no_refs)
    except Exception as e:
        print(f"[PDF ERR all] {path.name}: {e}")
        return ""

def read_pdf_until_keywords(path: Path, max_pages: int = 1) -> str:
    """
    Extract text from the beginning of the PDF up to the 'Keywords' section
    (accepts variations like 'Key words', 'Keyword', 'Key word').
    If 'Keywords' is not found, extracts only the first `max_pages` pages.
    This should include title, abstract, and keywords.
    """
    from pdfminer.high_level import extract_text
    import re

    try:
        txt = extract_text(str(path), page_numbers=set(range(max_pages))) or ""
    except Exception as e:
        print(f"[PDF ERR partial] {path.name}: {e}")
        return ""

    txt = re.sub(r"\s+", " ", txt).strip()

    # Regex pattern to detect 'Keywords' variants
    kw_pattern = re.compile(r"\bkey[\s\-*]words?\b[:\---]?", flags=re.
    IGNORECASE)
    m = kw_pattern.search(txt)
    if m:
        txt = txt[:m.end()]
    return txt

```

```

def split_sentences(text: str) -> List[str]:
    """Split text into sentences using punctuation as boundaries."""
    if not text:
        return []
    sentences = re.split(r"(?<=[\.\?\!\,])\s+", text)
    return [s.strip() for s in sentences if len(s.strip()) > 3]

def term_to_regex(term: str) -> str:
    """
    Regex robusta: spazi -> [\s-]+, match case-insensitive con \b...\b.
    """
    t = term.strip()
    t = re.escape(t)
    t = t.replace(r"\ ", r"[\s-]+").replace(r"\-", r"[\s-]+")
    return r"\b" + t + r"\b"

def compile_tool_patterns(tool_terms: Dict[str, List[str]]) -> Dict[str, re.
    Pattern]:
    patterns = {}
    for tool, terms in tool_terms.items():
        term_patterns = [term_to_regex(t) for t in terms]
        pat = r"(?i)(" + "|".join(term_patterns) + r")"
        patterns[tool] = re.compile(pat)
    return patterns

def cut_at_references(raw_text: str) -> str:
    """
    Cut the text at the section 'References' (or 'Bibliography') so as to
    exclude the list of references from the analysis.
    Look for a line that contains only 'References' (or 'Bibliography'),
    ignoring uppercase/lowercase.
    """
    import re
    if not raw_text:
        return ""

    pattern = re.compile(
        r"^\s*(references|bibliography)\s*[:\d\.]*\s*$",
        flags=re.IGNORECASE | re.MULTILINE
    )

    m = pattern.search(raw_text)
    if m:
        # cut after 'References'
        return raw_text[:m.start()]

```

```
return raw_text
```

```
[102]: # Compile regex patterns for all tools
patterns = compile_tool_patterns(tool_dict)

# SCAN PDF FILES

pdf_files = sorted(BASE_FOLDER.glob("*.pdf"))
rows: List[Dict[str, Any]] = []

# Loop through all PDF files and extract matches
for pdf in tqdm(pdf_files, desc="Scanning PDFs"):
    # Extract full text and front-matter (title, abstract, keywords)
    text_full = read_pdf_text(pdf)
    text_front = read_pdf_until_keywords(pdf)

    # Split text into sentences
    sentences_full = split_sentences(text_full)
    sentences_front = split_sentences(text_front)

    # Create a dictionary for the current PDF
    row: Dict[str, Any] = {"pdf_file": pdf.name}

    # Loop through all tools and check for matches
    for tool, rx in patterns.items():
        # Search only in front-matter for Meta-analysis and journal name
        if tool in {"Meta-analysis", "Trends in Food Science & Technology"}:
            sentences_source = sentences_front
        else:
            sentences_source = sentences_full

        # Find all sentences that match the pattern
        matched_sentences = [s for s in sentences_source if rx.search(s)]
        found = len(matched_sentences) > 0

        # Store up to 5 unique matched sentences
        uniq, seen = [], set()
        for s in matched_sentences:
            ss = s.strip()
            if ss not in seen:
                seen.add(ss)
                uniq.append(ss)
            if len(uniq) >= 5:
                break

        # Save results in the row dictionary
        row[f"{tool}_found"] = bool(found)
```

```

row[f"{tool}_sentences"] = " ".join(uniq) if uniq else ""

rows.append(row)

```

Scanning PDFs: 100% | 633/633 [38:50<00:00, 3.68s/it]

```

[125]: # CREATE DATAFRAME AND SAVE RESULTS

# Define column order: filename, then *_found flags, then *_sentences
cols = ["pdf_file"]
for tool in tool_dict.keys():
    cols.append(f"{tool}_found")
for tool in tool_dict.keys():
    cols.append(f"{tool}_sentences")

# Build the DataFrame
df = pd.DataFrame(rows)
df = df.reindex(columns=cols)

# Display and export results
#display(df)
df.to_csv(OUTPUT_CSV, index=False)
print("CSV saved to:", OUTPUT_CSV)

```

CSV saved to: /Users/martabonioli/Library/CloudStorage/Dropbox/Smart_labels/Response_letter_tothe_editor/Marta/relevant papers/scopus_consumer*systematicLitRev*food/RoB_QualityAss_analysis_scopus_633_keywords_271125_6.csv

1.0.2 CHECK IF META-ANALYSIS IS IN THE TITLE

and check Inspect mismatches (es. First page says meta-analysis, but title does NOT)

```

[87]: # CHECK IF META-ANALYSIS IS IN THE TITLE
# Robust filename pattern for meta-analysis variants ---
# Covers: "meta analysis", "meta-analysis", "metaanalysis", "meta_analyses",
# en/em dashes, and the common typo "analysi".
meta_in_title_pattern = re.compile(r"meta\[s_\]---]*analys(?:is|es|i)\b", re.
    IGNORECASE)
df["Meta_in_title"] = df["pdf_file"].str.contains(meta_in_title_pattern,
    regex=True, na=False)

# Define the two booleans we want to compare ---
# A) Found on FIRST PAGE (from your earlier pipeline): "Meta-analysis_found"
meta_first_page = df["Meta-analysis_found"].fillna(False)

# B) Found in the TITLE (filename): "Meta_in_title"
meta_title = df["Meta_in_title"].fillna(False)

```



```

# Helpful counts
TP = int(((meta_first_page == True) & (meta_title == True)).sum()) # both true
FP = int(((meta_first_page == True) & (meta_title == False)).sum()) #
    ↳first-page true, title false
FN = int(((meta_first_page == False) & (meta_title == True)).sum()) #
    ↳first-page false, title true
TN = int(((meta_first_page == False) & (meta_title == False)).sum()) # both
    ↳false

total = len(df)
print(f"\nTotals (n={total}):")
print(f"  Both TRUE: {TP}")
print(f"  FirstPage TRUE, title false: {FP}")
print(f"  Title TRUE, title true: {FN}")
print(f"  Not contain Meta-analysis: {TN}")

# Inspect mismatches ---
# First page says meta-analysis, but title does NOT
mismatch_fp = df.loc[(meta_first_page) & (~meta_title), ["pdf_file",
    ↳"Meta-analysis_sentences"]].copy()
mismatch_fp.rename(columns={"Meta-analysis_sentences": "FirstPage_matches"},
    ↳inplace=True)
print("\nMismatches: First page = TRUE, Title = FALSE")
#display(mismatch_fp.head(20))

# Title says meta-analysis, but first page does NOT
mismatch_fn = df.loc[(~meta_first_page) & (meta_title), ["pdf_file",
    ↳"Meta-analysis_sentences"]].copy()
mismatch_fn.rename(columns={"Meta-analysis_sentences": "FirstPage_matches"},
    ↳inplace=True)
print("\nMismatches: First page = FALSE, Title = TRUE")
display(mismatch_fn.head(20))

```

Totals (n=633):

Both TRUE:	18
FirstPage TRUE, title false:	33
Title TRUE, title true:	1
Not contain Meta-analysis:	581

Mismatches: First page = TRUE, Title = FALSE

Mismatches: First page = FALSE, Title = TRUE

	pdf_file	FirstPage_matches
565	The-concentration-of-pesticides-in-tomato-a-gl...	

1.0.3 SELECT NOT META-ANALYSIS ARTICLES

```
[88]: # SELECT NOT META-ANALYSIS ARTICLES

# create mask: no meta-analysis in title AND no meta-analysis on first page
mask_no_meta_anywhere = (~df["Meta_in_title"]) & (~meta_first_page)

# create the new filtered DataFrame ---
df_no_meta = df.loc[mask_no_meta_anywhere].copy()

print(f"Number of papers with NO meta-analysis (title nor first page): {len(df_no_meta)} out of {len(df)}")
print(f"Number of papers with meta-analysis (title nor first page): {len(df) - len(df_no_meta)}")
display(df_no_meta[["pdf_file", "Meta-analysis_found", "Meta_in_title"]])
```

Number of papers with NO meta-analysis (title nor first page): 581 out of 633
 Number of papers with meta-analysis (title nor first page): 52

	pdf_file	Meta-analysis_found	\
0	A Systematic Review of Salt Reduction Initiati...	False	
1	A Systematic Review of Trans Fat Reduction Ini...	False	
3	A comprehensive review of probiotic claims reg...	False	
4	A comprehensive systematic review and health r...	False	
5	A critical review on food loss and waste quant...	False	
..	...	...	
628	Why-do-consumers-make-green-purchase-decisions...	False	
629	characteristics_of_food_environments_that.7.pdf	False	
630	doption of Geographical Indications and origin...	False	
631	eFood - 2024 - Kumar - A systematic review exp...	False	
632	revention of childhood obesity through appropri...	False	

	Meta_in_title
0	False
1	False
3	False
4	False
5	False
..	...
628	False
629	False
630	False
631	False
632	False

[581 rows x 3 columns]

SELECT NOT META-ANALYSIS ARTICLES that use PRISMA This summary provides a quick overview of the methodological instruments reported by PRISMA-based literature reviews, allowing to see which tools are most commonly used across the dataset.

```
[89]: # Filter: only paper with PRISMA_found=True
mask_prisma = (df_no_meta["PRISMA_found"] == True)

# Apply filter
df_prisma_noMetaAna = df_no_meta[mask_prisma]

# tool columns
tool_cols = [c for c in df.columns if c.endswith("_found")]

summary = (
    df_prisma_noMetaAna[tool_cols]
    .sum() # somma dei True
    .reset_index()
    .rename(columns={"index": "Tool", 0: "Count"})
)

# Rimuove "_found" per estetica
summary["Tool"] = summary["Tool"].str.replace("_found", "", regex=False)

display(summary)
```

	Tool	Count
0	Quality assessment	97
1	Impact Factor	19
2	PRISMA	294
3	Peer review	217
4	Risk of Bias	98
5	Agreement	112
6	Disagreement	61
7	English	253
8	Meta-analysis	0
9	Trends in Food Science & Technology	10

“Agreement” = TRUE

```
[90]: # check papers with "Agreement" = TRUE
agreement = df_prisma_noMetaAna[df_prisma_noMetaAna["Agreement_found"] == True]

#print("112 articles that mention 'Agreement':\n")
#for idx, row in agreement.iterrows():
#    print(f"PDF: {row['pdf_file']}")
#    print(f"Sentences: {row['Agreement_sentences']}")
#    print("-" * 80)
```

Disagreement = TRUE

```
[91]: # check papers with "Disagreement" = TRUE
disagreement = df_prisma_noMetaAna[df_prisma_noMetaAna["Disagreement_found"] ==
↳ True]

#print("1 articles that mention 'Disagreement':\n")
#for idx, row in disagreement.iterrows():
#    print(f"PDF: {row['pdf_file']}")
#    print(f"Sentences: {row['Disagreement_sentences']}")
#    print("-" * 80)
```

This code analyzes a dataset of studies (PRISMA – NOT Meta-analysis) to determine how many papers include specific methodological or reporting components. Each component is stored in the dataset as a Boolean column (True/False), indicating whether the information was found in each study. 1. Reading Boolean Columns: Each line extracts a Boolean variable from the dataframe, representing whether a study includes the keywords define in the dictionary. Missing values (NaN) are converted to False, assuming that if the information is not present, it was not reported. 2. Counting Occurrences 3. Computing Combined Metrics: the code then calculates how many studies include combinations of methodological components. - qa & rob: studies reporting both QA and RoB - qa | rob: studies reporting at least one of QA or RoB - qa | rob | impact: studies with QA or RoB or impact factor - ...

This analysis provides a detailed overview of the methodological rigor of the included reviews, showing how often they report conducting quality assessment, risk-of-bias evaluation, and agreement/disagreement procedures among reviewers.

```
[130]: # === READ BOOLEAN COLUMNS ===
qa = df_prisma_noMetaAna["Quality assessment_found"].fillna(False)
rob = df_prisma_noMetaAna["Risk of Bias_found"].fillna(False)
agr = df_prisma_noMetaAna["Agreement_found"].fillna(False)
disagr = df_prisma_noMetaAna["Disagreement_found"].fillna(False)
peer_rev = df_prisma_noMetaAna["Peer review_found"].fillna(False)
impact = df_prisma_noMetaAna["Impact Factor_found"].fillna(False)

# === COUNTS ===
n_total = len(df_prisma_noMetaAna)

n_qa    = qa.sum()
n_rob   = rob.sum()
n_agr   = agr.sum()
n_disagr = disagr.sum()
n_peer_rev = peer_rev.sum()
n_impact = impact.sum()

n_union_QA_impact = (qa | impact).sum()
n_both_QA_ROB = (n_union_QA_impact & rob).sum()
n_union_QA_ROB = (qa | rob).sum()
n_union_QA_ROB_impact = (qa | rob | impact).sum()
```

```

n_both_DIS_AGR = (disagr & agr).sum()
n_union_DIS_AGR = (disagr | agr).sum()
n_union_DIS_AGR_impact = (disagr | agr | impact).sum()

#NON (QA o ROB) MA (AGR o DISAGR)
mask_only_AGR_DIS_no_QA_ROB = ~(qa | rob | impact) & (agr | disagr)
n_only_AGR_DIS_no_QA_ROB = mask_only_AGR_DIS_no_QA_ROB.sum()

# HA (QA o ROB) MA (AGR o DISAGR)
mask_only_AGR_DIS_QA_ROB = (qa | rob) & (agr | disagr)
n_only_AGR_DIS_QA_ROB = mask_only_AGR_DIS_QA_ROB.sum()

# === PRINT RESULTS ===
print(f"PRISMA & NOT Meta-analysis: total = {n_total}\n")

print(f"  With Quality assessment:    {n_union_QA_impact}")
print(f"  With Risk of Bias:             {n_rob}")
print(f"  With Agreement:                  {n_agr}")
print(f"  With Disagreement:               {n_disagr}")
print(f"  Peer review:                     {n_peer_rev}")
#print(f"  Impact Factor:                  {n_impact}")

print(f"\n  With BOTH QA & RoB:              {n_both_QA_ROB}")
print(f"  With AT LEAST ONE (QA or RoB): {n_union_QA_ROB}")
print(f"  With AT LEAST ONE (QA or RoB or impact): {n_union_QA_ROB_impact}")

print(f"\n  With BOTH AGR & DISAGR:          {n_both_DIS_AGR}")
print(f"  With AT LEAST ONE (AGR or DISAGR): {n_union_DIS_AGR}")
print(f"  With AT LEAST ONE (AGR or DISAGR or impact): ↵
    ↵{n_union_DIS_AGR_impact}")

print(f"\n  ONLY AGR/DISAGR AND NO QA/ROB:    {n_only_AGR_DIS_no_QA_ROB}")
print(f"  ONLY AGR/DISAGR AND QA/ROB:       {n_only_AGR_DIS_QA_ROB}")

# === PERCENTAGES ===
print("\nPercentages:")
if n_total > 0:
    print(f"  - Quality Assessment: {n_union_QA_impact / n_total * 100:.1f}%")
    print(f"  - Risk of Bias: {n_rob / n_total * 100:.1f}%")
    print(f"  - Agreement: {n_agr / n_total * 100:.1f}%")
    print(f"  - Disagreement: {n_disagr / n_total * 100:.1f}%")
    print(f"  - Peer review: {n_peer_rev / n_total * 100:.1f}%")
    print(f"  - Impact Factor: {n_impact / n_total * 100:.1f}%")
    print(f"  - AT LEAST ONE (QA or RoB): {n_union_QA_ROB / n_total * 100:.1f}%")

```

```

    print(f" - With AT LEAST ONE (QA or RoB or impact): {n_union_QA_ROB_impact/
↪n_total * 100:.1f}%")
    print(f" - AT LEAST ONE (AGR or DISAGR): {n_union_DIS_AGR / n_total * 100:.
↪1f}%")
    print(f" - With AT LEAST ONE (AGR or DISAGR or impact):_
↪{n_union_DIS_AGR_impact/ n_total * 100:.1f}%")
    print(f" - ONLY AGR/DISAGR AND NO QA/ROB: : {n_only_AGR_DIS_no_QA_ROB /_
↪n_total * 100:.1f}%")
    print(f" - ONLY AGR/DISAGR AND QA/ROB: : {n_only_AGR_DIS_QA_ROB / n_total *_
↪100:.1f}%")

else:
    print(" - No papers match the filter (n_total = 0).")

```

PRISMA & NOT Meta-analysis: total = 294

With Quality assessment:	109
With Risk of Bias:	98
With Agreement:	112
With Disagreement:	61
Peer review:	217

With BOTH QA & RoB:	98
With AT LEAST ONE (QA or RoB):	146
With AT LEAST ONE (QA or RoB or impact):	157

With BOTH AGR & DISAGR:	39
With AT LEAST ONE (AGR or DISAGR):	134
With AT LEAST ONE (AGR or DISAGR or impact):	145

ONLY AGR/DISAGR AND NO QA/ROB:	45
ONLY AGR/DISAGR AND QA/ROB:	84

Percentages:

- Quality Assessment: 37.1%
- Risk of Bias: 33.3%
- Agreement: 38.1%
- Disagreement: 20.7%
- Peer review: 73.8%
- Impact Factor: 6.5%
- AT LEAST ONE (QA or RoB): 49.7%
- With AT LEAST ONE (QA or RoB or impact): 53.4%
- AT LEAST ONE (AGR or DISAGR): 45.6%
- With AT LEAST ONE (AGR or DISAGR or impact): 49.3%
- ONLY AGR/DISAGR AND NO QA/ROB: : 15.3%
- ONLY AGR/DISAGR AND QA/ROB: : 28.6%

[]:

1.0.4 SUBSET OF ARTICLES published in Trends in Food Science & Technology

```
[94]: mask_journal = (  
    (df["PRISMA_found"] == True)  
    & (df["Meta-analysis_found"] == False)  
    & (df["Trends in Food Science & Technology_found"] == True)  
)  
  
subset = df.loc[mask_journal,  
    [  
        "pdf_file",  
        "Risk of Bias_found",  
        "Quality assessment_found",  
    ],  
]  
display(subset)
```

	pdf_file	Risk of Bias_found \
137	Consumer behaviour toward "smart" food labels-...	False
186	Digital nudging in online grocery stores- A sc...	True
210	Eat or what to eat- A systematic review of foo...	True
252	Factors affecting consumers' evaluation of foo...	False
472	Regional analysis in consumer preferences for ...	False
477	Research for the retail grocery context- A sys...	False
501	Social media and food consumer behavior- A sys...	True
503	Socialfood- Virtuous or vicious? A systematic ...	False
535	The application of virtual reality in food con...	False
583	Thirty years of knowledge on sourdough ferment...	False

	Quality assessment_found
137	False
186	False
210	False
252	True
472	False
477	False
501	False
503	False
535	False
583	False

1.1 ANALYSIS OF META-ANALYSIS articles

```
[103]: # ONLY META-ANALYSIS
# Mask for papers WITH meta-analysis (either in title OR on the first page)
mask_meta_anywhere = (df["Meta_in_title"]) | (meta_first_page)

# Create the filtered DataFrame
df_meta = df.loc[mask_meta_anywhere].copy()

# Report counts
print(f"Number of papers WITH meta-analysis (title or first page):_
↪{len(df_meta)} out of {len(df)}")
```

Number of papers WITH meta-analysis (title or first page): 52 out of 633

```
[104]: # Filter: only paper with PRISMA_found=True
mask_prisma = (df_meta["PRISMA_found"] == True)

# Apply filter
df_prisma_MetaAna = df_meta[mask_prisma]

# tool columns
tool_cols = [c for c in df.columns if c.endswith("_found")]

summary = (
    df_prisma_MetaAna[tool_cols]
    .sum() # somma dei True
    .reset_index()
    .rename(columns={"index": "Tool", 0: "Count"})
)

# Rimuove "_found" per estetica
summary["Tool"] = summary["Tool"].str.replace("_found", "", regex=False)

display(summary)
```

	Tool	Count
0	Quality assessment	22
1	Impact Factor	0
2	PRISMA	41
3	Peer review	19
4	Risk of Bias	31
5	Agreement	10
6	Disagreement	13
7	English	27
8	Meta-analysis	40
9	Trends in Food Science & Technology	0


```

[121]: # === READ BOOLEAN COLUMNS ===
qa_meta = df_prisma_MetaAna["Quality assessment_found"].fillna(False)
rob_meta = df_prisma_MetaAna["Risk of Bias_found"].fillna(False)
agr_meta = df_prisma_MetaAna["Agreement_found"].fillna(False)
disagr_meta = df_prisma_MetaAna["Disagreement_found"].fillna(False)
peer_rev_meta = df_prisma_MetaAna["Peer review_found"].fillna(False)
impact_meta = df_prisma_MetaAna["Impact Factor_found"].fillna(False)

# === COUNTS ===
n_total_meta = len(df_prisma_MetaAna)

n_qa_meta = qa_meta.sum()
n_rob_meta = rob_meta.sum()
n_agr_meta = agr_meta.sum()
n_disagr_meta = disagr_meta.sum()
n_peer_rev_meta = peer_rev_meta.sum()
n_impact_meta = impact_meta.sum()

n_both_QA_ROB_meta = (qa_meta & rob_meta).sum()
n_union_QA_ROB_meta = (qa_meta | rob_meta).sum()
n_union_QA_ROB_impact_meta = (qa_meta | rob_meta | impact_meta).sum()

n_both_DIS_AGR_meta = (disagr_meta & agr_meta).sum()
n_union_DIS_AGR_meta = (disagr_meta | agr_meta).sum()
n_union_DIS_AGR_impact_meta = (disagr_meta | agr_meta | impact_meta).sum()

#NON (QA o ROB) MA (AGR o DISAGR)
mask_only_AGR_DIS_no_QA_ROB_meta = ~(qa_meta | rob_meta) & (agr_meta |
    ↪disagr_meta)
n_only_AGR_DIS_no_QA_ROB_meta = mask_only_AGR_DIS_no_QA_ROB_meta.sum()

# HA (QA o ROB) MA (AGR o DISAGR)
mask_only_AGR_DIS_QA_ROB_meta = (qa_meta | rob_meta) & (agr_meta | disagr_meta)
n_only_AGR_DIS_QA_ROB_meta = mask_only_AGR_DIS_QA_ROB_meta.sum()

# === PRINT RESULTS ===
print(f"PRISMA & Meta-analysis: total = {n_total_meta}\n")

print(f"  With Quality assessment:    {n_qa_meta}")
print(f"  With Risk of Bias:              {n_rob_meta}")
print(f"  With Agreement:                  {n_agr_meta}")
print(f"  With Disagreement:               {n_disagr_meta}")

print(f"\n  With BOTH QA & RoB:                {n_both_QA_ROB_meta}")
print(f"  With AT LEAST ONE (QA or RoB): {n_union_QA_ROB_meta}")

```

```

print(f"  With AT LEAST ONE (QA or RoB or impact):␣
↪{n_union_QA_ROB_impact_meta}")

print(f"\n  With BOTH AGR & DISAGR:          {n_both_DIS_AGR_meta}")
print(f"  With AT LEAST ONE (AGR or DISAGR): {n_union_DIS_AGR_meta}")
print(f"  With AT LEAST ONE (AGR or DISAGR or impact):␣
↪{n_union_DIS_AGR_impact_meta}")

print(f"\n  ONLY AGR/DISAGR AND NO QA/ROB:      {n_only_AGR_DIS_no_QA_ROB_meta}")
print(f"  ONLY AGR/DISAGR AND QA/ROB:          {n_only_AGR_DIS_QA_ROB_meta}")

# === PERCENTAGES ===
print("\nPercentages:")
if n_total > 0:
    print(f" - Quality Assessment: {n_qa_meta / n_total_meta * 100:.1f}%")
    print(f" - Risk of Bias: {n_rob_meta / n_total_meta * 100:.1f}%")
    print(f" - Agreement: {n_agr_meta / n_total_meta * 100:.1f}%")
    print(f" - Disagreement: {n_disagr_meta / n_total_meta * 100:.1f}%")
    print(f" - AT LEAST ONE (QA or RoB): {n_union_QA_ROB_meta / n_total_meta *␣
↪100:.1f}%")
    print(f" - AT LEAST ONE (AGR or DISAGR): {n_union_DIS_AGR_meta /␣
↪n_total_meta * 100:.1f}%")

    print(f" - With AT LEAST ONE (QA or RoB or impact):␣
↪{n_union_QA_ROB_impact_meta/ n_total_meta * 100:.1f}%")
    print(f" - With AT LEAST ONE (AGR or DISAGR or impact):␣
↪{n_union_DIS_AGR_impact_meta/ n_total_meta * 100:.1f}%")

    print(f" - ONLY AGR/DISAGR AND NO QA/ROB: : {n_only_AGR_DIS_no_QA_ROB_meta /␣
↪n_total_meta * 100:.1f}%")
    print(f" - ONLY AGR/DISAGR AND QA/ROB: : {n_only_AGR_DIS_QA_ROB_meta /␣
↪n_total_meta * 100:.1f}%")

else:
    print(" - No papers match the filter (n_total = 0).")

```

PRISMA & Meta-analysis: total = 41

With Quality assessment:	22
With Risk of Bias:	31
With Agreement:	10
With Disagreement:	13

With BOTH QA & RoB:	21
With AT LEAST ONE (QA or RoB):	32
With AT LEAST ONE (QA or RoB or impact):	32

With BOTH AGR & DISAGR: 5
 With AT LEAST ONE (AGR or DISAGR): 18
 With AT LEAST ONE (AGR or DISAGR or impact): 18

ONLY AGR/DISAGR AND NO QA/ROB: 1
 ONLY AGR/DISAGR AND QA/ROB: 17

Percentages:

- Quality Assessment: 53.7%
- Risk of Bias: 75.6%
- Agreement: 24.4%
- Disagreement: 31.7%
- AT LEAST ONE (QA or RoB): 78.0%
- AT LEAST ONE (AGR or DISAGR): 43.9%
- With AT LEAST ONE (QA or RoB or impact): 78.0%
- With AT LEAST ONE (AGR or DISAGR or impact): 43.9%
- ONLY AGR/DISAGR AND NO QA/ROB: : 2.4%
- ONLY AGR/DISAGR AND QA/ROB: : 41.5%

[]:

1.2 Comparison of the percentages of studies that applied PRISMA-related quality tools between:

1) Studies that are not meta-analyses

2) Studies that are meta-analyses For each group, the code computes the percentage of papers that used: Quality Assessment tools, Risk of Bias tools, At least one of the two tools

```
[124]: def pct(n, total):
    if total and total > 0:
        return n / total * 100
    return np.nan

rows = [
    ("Quality Assessment",          pct(n_qa, n_total),
    ↪      pct(n_qa_meta, n_total_meta)),
    ("Risk of Bias",               pct(n_rob, n_total),
    ↪      pct(n_rob_meta, n_total_meta)),
    ("Agreement",                  pct(n_agr, n_total),
    ↪      pct(n_agr_meta, n_total_meta)),
    ("Disagreement",               pct(n_disagr, n_total),
    ↪      pct(n_disagr_meta, n_total_meta)),
    ("Peer Review",                 pct(n_peer_rev, n_total),
    ↪      pct(n_peer_rev_meta, n_total_meta)),
    ("Impact Factor",               pct(n_impact, n_total),
    ↪      pct(n_impact_meta, n_total_meta)),
```

```

    ("AT LEAST ONE (QA or RoB)",          pct(n_union_QA_ROB, n_total),
    ↪          pct(n_union_QA_ROB_meta, n_total_meta)),
    ("AT LEAST ONE (AGR or DISAGR)",      pct(n_union_DIS_AGR, n_total),
    ↪          pct(n_union_DIS_AGR_meta, n_total_meta)),
    ("(AGR or DISAGR) AND (NO QA or ROB)",
    ↪          pct(n_only_AGR_DIS_no_QA_ROB, n_total),
    ↪          pct(n_only_AGR_DIS_no_QA_ROB_meta, n_total_meta)),
    ("(AGR or DISAGR) AND (QA or ROB)",    pct(n_only_AGR_DIS_QA_ROB,
    ↪          n_total),          pct(n_only_AGR_DIS_QA_ROB_meta, n_total_meta)),

    ("AT LEAST ONE (QA or RoB or Impact)", pct(n_union_QA_ROB_impact,
    ↪          n_total),          pct(n_union_QA_ROB_impact_meta, n_total_meta)),
    ("AT LEAST ONE (AGR or DISAGR or Impact)", pct(n_union_DIS_AGR_impact,
    ↪          n_total),          pct(n_union_DIS_AGR_impact_meta, n_total_meta)),
]

df_compare = pd.DataFrame(rows, columns=["Metric", "NO META-ANALYSIS (%)",
    ↪          "META-ANALYSIS (%)"])

for col in ["NO META-ANALYSIS (%)", "META-ANALYSIS (%)"]:
    df_compare[col] = df_compare[col].apply(lambda x: f"{x:.1f}%" if pd.
    ↪          notnull(x) else "NA")

print(df_compare.to_string(index=False))

```

	Metric	NO META-ANALYSIS (%)	META-ANALYSIS (%)
	Quality Assessment	33.0%	53.7%
	Risk of Bias	33.3%	75.6%
	Agreement	38.1%	24.4%
	Disagreement	20.7%	31.7%
	Peer Review	73.8%	46.3%
	Impact Factor	6.5%	0.0%
	AT LEAST ONE (QA or RoB)	49.7%	78.0%
	AT LEAST ONE (AGR or DISAGR)	45.6%	43.9%
	(AGR or DISAGR) AND (NO QA or ROB)	17.0%	2.4%
	(AGR or DISAGR) AND (QA or ROB)	28.6%	41.5%
	AT LEAST ONE (QA or RoB or Impact)	53.4%	78.0%
	AT LEAST ONE (AGR or DISAGR or Impact)	49.3%	43.9%

2 Classify literature reviews by language inclusion/exclusion criteria

This code evaluates the inclusion/exclusion criteria used by each literature review regarding the languages of the studies they consider.

- English-only criteria (English_found = True and Other_languages_found = False)

- Acceptance of studies in multiple languages (English_found = True and Other languages_found = True)
- No explicit language criteria mentioned

The function `classify_language_criteria()` assigns each review to one of these categories. Finally, the script calculates and prints both counts and percentages for each group.

This allows you to quantify how many literature reviews restrict their included studies to English only, and how many include studies written in other languages.

```
[114]: # Filter only papers with English_found == True
df_english = df[df["English_found"] == True].copy()

# Count how many
n_english = len(df_english)
print(f"Number of papers with English_found = True: {n_english}")

# Save to Excel
output_excel = "papers_in_english_633.xlsx"
df_english.to_excel(output_excel, index=False)

print("Excel file saved as:", output_excel)
```

Number of papers with English_found = True: 458

Excel file saved as: papers_in_english_633.xlsx

```
[115]: other_language_terms = [
    "spanish", "español",
    "japanese", " ",
    "chinese", " ", "mandarin",
    "arabic", " ",
    "russian", " ",
    "french", "français",
    "german", "deutsch",
    "portuguese", "português",
    "italian", "italiano",
    "korean", " ",
    "dutch", "nederlands",
    "swedish", "svenska"
]
```

```
[116]: def contains_other_language(sentence):
    if not isinstance(sentence, str):
        return False
    s = sentence.lower()
    return any(lang.lower() in s for lang in other_language_terms)

# NEW COLUMN: True if ANY sentence mentions another language
```

```
df_english["Other_languages_found"] = df_english["English_sentences"].
    ↪ apply(contains_other_language)
```

```
[117]: n_total_english = len(df_english)
n_also_other = df_english["Other_languages_found"].sum()

print(f"Total papers with English_found = True: {n_total_english}")
print(f"Papers that also mention other languages: {n_also_other} ({n_also_other_
    ↪ / n_total_english * 100:.1f}%)")
```

Total papers with English_found = True: 458
Papers that also mention other languages: 58 (12.7%)

```
[118]: examples = df_english[df_english["Other_languages_found"] == True].head(5)

print("Five examples of papers that mention other languages:\n")
for idx, row in examples.iterrows():
    print(f"PDF: {row['pdf_file']}")
    print(f"Sentences: {row['English_sentences']}")
    print("-" * 80)
```

Five examples of papers that mention other languages:

PDF: A Systematic Review of Trans Fat Reduction Initiatives in the Eastern Mediterranean Region.pdf

Sentences: Only articles published after 1995, in English, Arabic or French, were included. Only English, Arabic and French languages were considered. The search was limited to materials published post 1995 in English, Arabic and French languages only. Individual articles were also excluded if they were published before 1995, or in any language besides English, Arabic, or French.

PDF: A systematic review of the evaluation of agricultural policies- Using prisma .pdf

Sentences: Database Scopus Science Direct Web of Science Dimension EBSCO Dialnet SciELO Coverage 1974-2021 1977-2022 2016-2022 1970-2022 1966-2022 - - Language English English English English English, Spanish, Arabic, Russian - - Heliyon9(2023)e202924 L.M.

PDF: A systematic review of the nutrient composition, microbiological and toxicological profile of Acheta domesticus (house cricket).pdf

Sentences: The majority of publications screened was in English, but we also identified articles in other languages (e.g., French, German).

PDF: A-Systematic-Review-of-the-Scope-and-Patterns-of-Green-Consumption-in-SubSaharan-Africa_2023_MDPI.pdf

Sentences: The review was conducted in English and French, i.e., the two most widely spoken languages in sub-Saharan Africa. They also thank Isis Olivier

for her review of this article in English.

PDF: A-review-of-the-literature-on-food-values-and-their-potential-implications-for-consumers-food-decision-processes_2022_MDPI.pdf

Sentences: Language: English; Years of publication: 2009-2021. Conducting Stage of the Review In this stage, we applied the following criteria to the Web of Science (WoS) and Scopus databases: The articles had to be published in English between 2009 and 2021. He showed that food values vary based on participants' language use, as English speakers were far more likely than Spanish speakers to report that convenience is a very important reason for preferring fast food to home cooking; Spanish-speaking participants placed greater importance on price, nutrition, ease of preparation, and food longevity when choosing between food items. These findings could be questioned due to the economic differences that may arise between English speakers and Spanish speakers, thus making income an important factor [25].

```
[119]: output_excel = "english_papers_with_other_languages_58.xlsx"
df_english.to_excel(output_excel, index=False)

print("Excel file saved as:", output_excel)
```

Excel file saved as: english_papers_with_other_languages_58.xlsx